Домашнее задание 1

Important notice

1) **Это задание выполняется и оценивается индивидуально или в группах до 4 человек!** Если вы используете код из открытых источников (репозитории ваших одногруппников таковыми не считаются) - пожалуйста, указывайте ссылки на них (можно в readme вести лог всех источников, откуда вы берете код). Находить готовые рецепты в интернете - хорошо, списывать - плохо.
2) **При обнаружении списывания (одинаковый код в двух репозиториях без указания внешнего источника) оценка будет выставляться студенту (группе), чей коммит с решением был первый**. Остальным - 0 баллов и докладная в УО.
3) Чтобы исключить возможность списывания, **рекомендуется сделать ваш репозиторий с домашним заданием приватным**.

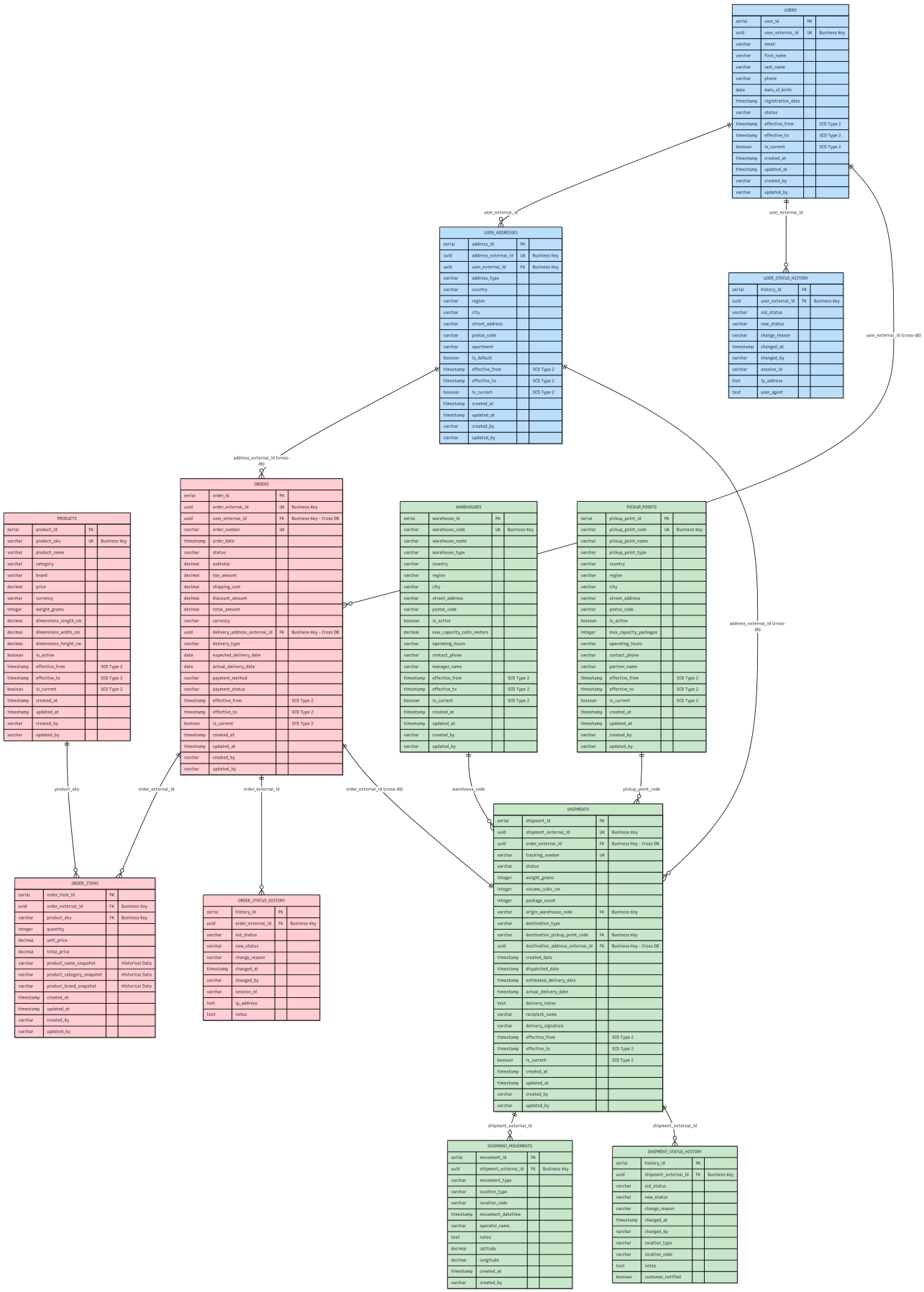
Формулировка

У нас есть 3 микросервиса, которые работают внутри большого маркетплейса:

- **user_service** - содержит информацию о пользователях
- **order_service** - содежит информацию о заказах пользователей
- **logistics_service** - содержит информацию о том, как заказ разбился на посылки и как они перемещаются по логистической сети (от складов к ПВЗ)

Их базы данных выглядят следующим образом:

- **user_service_db** - отмечена синим
- **order_service_db** - отмечена красным
- **logistics_service_db** - отмечена зеленым


USERS
serial user_id PK
uuid user_external_id UK Business Key
varchar email
varchar first_name
varchar last_name
varchar phone
date date_of_birth
timestamp registration_date
varchar status
timestamp effective_from SCD Type 2
timestamp effective_to SCD Type 2
boolean is_current SCD Type 2
timestamp created_at
timestamp updated_at
varchar created_by
varchar updated_by
user_external_id
user_external_id
USER_STATUS_HISTORY
serial history_id PK
uuid user_external_id FK Business Key
varchar old_status
varchar new_status
varchar change_reason
timestamp changed_at
varchar changed_by
varchar session_id
inet ip_address
text user_agent
user_external_id (cross-db)
USER_ADDRESSES
serial address_id PK
uuid address_external_id UK Business Key
uuid user_external_id FK Business Key
varchar address_type
varchar country
varchar region
varchar city
varchar street_address
varchar postal_code
varchar apartment
boolean is_default
timestamp effective_from SCD Type 2
timestamp effective_to SCD Type 2
boolean is_current SCD Type 2
timestamp created_at
timestamp updated_at
varchar created_by
varchar updated_by
address_external_id (cross-db)
address_external_id (cross-db)
PRODUCTS
serial product_id PK
varchar product_sku UK Business Key
varchar product_name
varchar category
varchar brand
decimal price
varchar currency
integer weight_grams
decimal dimensions_length_cm
decimal dimensions_width_cm
decimal dimensions_height_cm
boolean is_active
timestamp effective_from SCD Type 2
timestamp effective_to SCD Type 2
boolean is_current SCD Type 2
timestamp created_at
timestamp updated_at
varchar created_by
varchar updated_by
ORDERS
serial order_id PK
uuid order_external_id UK Business Key
uuid user_external_id FK Business Key - Cross DB
varchar order_number UK
timestamp order_date
varchar status
decimal subtotal
decimal tax_amount
decimal shipping_cost
decimal discount_amount
decimal total_amount
varchar currency
uuid delivery_address_external_id FK Business Key - Cross DB
varchar delivery_type
date expected_delivery_date
date actual_delivery_date
varchar payment_method
varchar payment_status
timestamp effective_from SCD Type 2
timestamp effective_to SCD Type 2
boolean is_current SCD Type 2
timestamp created_at
timestamp updated_at
varchar created_by
varchar updated_by
WAREHOUSES
serial warehouse_id PK
varchar warehouse_code UK Business Key
varchar warehouse_name
varchar warehouse_type
varchar country
varchar region
varchar city
varchar street_address
varchar postal_code
boolean is_active
decimal max_capacity_cubic_meters
varchar operating_hours
varchar contact_phone
varchar manager_name
timestamp effective_from SCD Type 2
timestamp effective_to SCD Type 2
boolean is_current SCD Type 2
timestamp created_at
timestamp updated_at
varchar created_by
varchar updated_by
PICKUP_POINTS
serial pickup_point_id PK
varchar pickup_point_code UK Business Key
varchar pickup_point_name
varchar pickup_point_type
varchar country
varchar region
varchar city
varchar street_address
varchar postal_code
boolean is_active
integer max_capacity_packages
varchar operating_hours
varchar contact_phone
varchar partner_name
timestamp effective_from SCD Type 2
timestamp effective_to SCD Type 2
boolean is_current SCD Type 2
timestamp created_at
timestamp updated_at
varchar created_by
varchar updated_by
product_sku
order_external_id
order_external_id
order_external_id (cross-db)
warehouse_code
pickup_point_code
ORDER_ITEMS
serial order_item_id PK
uuid order_external_id FK Business Key
varchar product_sku FK Business Key
integer quantity
decimal unit_price
decimal total_price
varchar product_name_snapshot Historical Data
varchar product_category_snapshot Historical Data
varchar product_brand_snapshot Historical Data
timestamp created_at
timestamp updated_at
varchar created_by
varchar updated_by
ORDER_STATUS_HISTORY
serial history_id PK
uuid order_external_id FK Business Key
varchar old_status
varchar new_status
varchar change_reason
timestamp changed_at
varchar changed_by
varchar session_id
inet ip_address
text notes
SHIPMENTS
serial shipment_id PK
uuid shipment_external_id UK Business Key
uuid order_external_id FK Business Key - Cross DB
varchar tracking_number UK
varchar status
integer weight_grams
integer volume_cubic_cm
integer package_count
varchar origin_warehouse_code FK Business Key
varchar destination_type
varchar destination_pickup_point_code FK Business Key
uuid destination_address_external_id FK Business Key - Cross DB
timestamp created_date
timestamp dispatched_date
timestamp estimated_delivery_date
timestamp actual_delivery_date
text delivery_notes
varchar recipient_name
varchar delivery_signature
timestamp effective_from SCD Type 2
timestamp effective_to SCD Type 2
boolean is_current SCD Type 2
timestamp created_at
timestamp updated_at
varchar created_by
varchar updated_by
shipment_external_id
shipment_external_id
SHIPMENT_MOVEMENTS
serial movement_id PK
uuid shipment_external_id FK Business Key
varchar movement_type
varchar location_type
varchar location_code
timestamp movement_datetime
varchar operator_name
text notes
decimal latitude
decimal longitude
timestamp created_at
varchar created_by
SHIPMENT_STATUS_HISTORY
serial history_id PK
uuid shipment_external_id FK Business Key
varchar old_status
varchar new_status
varchar change_reason
timestamp changed_at
varchar changed_by
varchar location_type
varchar location_code
text notes
boolean customer_notified

Задача:
1. Поднять инстанс PostgreSQL в docker-compose;
2. Создать 3 файла с миграциями, которые создадут структуру трех БД выше (для учебных целей нас утроит, что все эти БД будут жить на одном физическом хосте PostgreSQL); **Важно**, чтобы БД / схемы / таблицы /поля назывались именно так, как указано на схеме - иначе ДЗ не пройдет автотесты;
   1. **user_service_db** -> схема public -> таблицы / constraints / keys
   2. **order_service_db** -> схема public -> таблицы / constraints / keys
   3. **logistics_service_db** -> схема public -> таблицы / constraints / keys
3. Настроить автоматическую инициализацию БД с данной структурой
4. Поднять реплику в docker-compose, настроить репликацию

Бонусные задания:
1. Используя базу данных **order_service_db**, провести **когортный анализ** клиентов для понимания их поведения во времени.
   Когорта - это группа клиентов, которые совершили первую покупку в одном временном периоде (месяце).
   1. Определение когорт:
      1. Определить **месяц первой покупки** для каждого клиента (когорту)
      2. Рассчитать **размер каждой когорты** (количество новых клиентов)
   2. Анализ активности когорт на горизонте следующего полугодия:
      1. Для каждой когорты рассчитать количество **активных клиентов** в каждый последующий месяц
      2. Рассчитать **Retention Rate** - долю клиентов, которые вернулись в каждый период
   3. Рассчитать метрики для каждой когорты:
      1. **Total Cohort Revenue**
      2. **Average Revenue Per Customer**
   4. По итогу должна получиться таблица вида:

```
cohort_month | cohort_size | period_0_pct | period_1_pct |
period_2_pct | period_3_pct | period_4_pct | period_5_pct |
total_cohort_revenue | avg_revenue_per_customer
```

2. Превратить скрипт выше во view
3. Сделать инсталляцию PostgreSQL отказоустойчивой (Patroni, etcd, HAProxy)

Важно:
- Связи между таблицами в разных БД - логические; Для них не нужно создавать FK (это у вас не должно в целом получиться)
- Активным считаем клиента, совершившего хотя бы 1 заказ
- Если необходимо — я вам сгенерировал ~100 Mb данных для тестов; https://clck.ru/3QCYgU ; Эти данные ТОЛЬКО для тестов, в домашку засылать/инитить с ними базу не нужно

Сроки

1. Дедлайн на 100% - 2 недели - **23.11.2025 23:59:59 включительно**
2. Дедлайн на 75% - до дедлайна следующего ДЗ
3. Дедлайн на 50% - до конца курса

Как сдавать ДЗ?

- Готовое ДЗ загружается на GitHub (в приватный репо + выдать мне доступ - @mgcrp)
- ДЗ №1 выполняется в ветке HW 1
- Загружайте только файлы конфигурации, код и markdown с описанием; не загружайте в GitHub бинари, архивы с кодом, видео и/или файлы pgdata; Для того, чтобы такие файлы не попадали в репозиторий, настройте себе .gitignore
- К репо должен быть приложен README.md с описанием того, что вы сделали и как это запустить
  - **Обязательно** должен быть указан connection_string к вашей БД или балансеру
- Задание сдается в форму: https://forms.gle/Yj2UTYcBQqhmBLL77

Критерии оценки

Балл	Критерий
4	Запускается только master (с помощью docker-compose), БД инициализирована с нужной структурой
8	Запускаются master и replica (с помощью docker-compose), обе БД инициализированы с нужной структурой, работает репликация
+2 балла	Написан SQL-скрипт, который считает витрину для когортного анализа
+1 балл	Скрипт из предыдущего пункта преобразован во view
+2 балла	Настроен HA-кластер PostgreSQL

Максимальный балл за ДЗ - 13

Как это будет проверяться?

1) Запуск проекта по инструкции из вашего readme.md
   - docker-compose up -d или bash-скрипт (если вам нужно сделать несколько действий сразу)
2) Проверка количества хостов
3) Проверка того, что replication slot активен, на мастер идет streaming, а на replica - receiving
4) Проверка структуры БД на хостах
5) **Автотест**: python-скрипт, который создает синтетические данные и пишет их в master, после проверяет, что все сгенеренные данные попали в реплику
6) Проверка бонусных скриптов и view на загруженных в базу синтетических данных